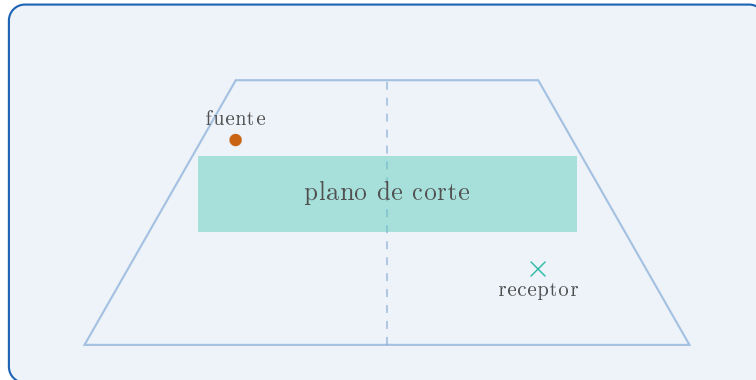


Prototipo 1

Modelador de Recintos 3D
con Simulación Acústica FEM

Manual de Usuario



Ingeniería en Sonido

Versión del Manual 2.22 — 13 de Agosto de 2026

con importación CAD y router de mallado best-effort

Índice

1. Introducción	2
2. Inicio rápido	2
3. Interfaz general	3
4. Diseño del recinto (pestaña Geometría)	3
4.1. Parámetros básicos	3
4.2. Tipos de techo	3
4.3. Inclinação de paredes	3
4.4. Polígono personalizado	4
4.5. Modos de visualización	4
5. Controles del visor 3D	4
5.1. Navegación	4
5.2. Interacciones acústicas	4
6. Módulo acústico (pestaña Acústica)	4
6.1. Fuentes omnidireccionales	4
6.1.1. Configuración de la fuente	5
6.2. Receptor	5
6.3. Materiales por cara (estilo EASE)	5
6.4. Parches de absorción sub-cara (v2.17)	8
7. Cálculo de modos FEM	9
7.1. Parámetros	9
7.2. Procedimiento	9
8. Visualización del campo acústico	10
8.1. Nube de puntos 3D	10
8.2. Plano de corte 2D — flujo interactivo	10
8.2.1. Mapa de calor 2D	11
9. Respuesta en Frecuencia (FRF)	11
9.1. Cálculo	11
9.2. Gráfico de FRF	11
9.3. Escucha — ruido rosa filtrado por la sala	11
10. RT60 y materiales — comparativa de métodos	12
10.1. Métodos de predicción	12
10.2. Diálogo comparativo	12
10.3. Código de colores	13
10.4. Amortiguamiento modal	13
11. Flujo de trabajo completo	14
12. Referencia rápida de atajos	14
13. Solución de problemas frecuentes	15

14. Conceptos físicos clave	15
14.1. Modos acústicos	15
14.2. Sensibilidad vs. caudal volumétrico Q	15
14.3. Amortiguamiento modal y RT60	16
14.4. Frecuencia de Schroeder	16
15. Importar CAD y motor de mallado	16
15.1. Por qué hay dos motores	16
15.2. El badge de estado	17
15.3. Importar un archivo CAD	17
15.4. Diálogo de reparación guiada	17
15.5. Override manual del motor	18
15.6. Formatos soportados	18
15.7. Persistencia en <code>.room v3</code>	18
15.8. Cuándo usa cada motor el router automático	19
15.9. Workflow recomendado para auditorios complejos	19
15.10 El sistema best-effort y el fallback	20
16. Escala y orientación al importar	21
16.1. Escala — unidades del archivo	21
16.2. Orientación — eje vertical del archivo	21
16.3. Tres botones del diálogo de escala (v2.8)	22
16.4. Optimizaciones del reparador de mallas	22
17. Indicador de ejes y rotación con eje fijo	22
17.1. Fijar un eje	22
17.2. Comportamiento de la rueda del mouse con eje fijo	23
18. Rendimiento y benchmarks	23
18.1. Cómo correr los benchmarks	23
18.2. Qué mide	24
18.3. Resultados de referencia	24
18.4. Cuellos de botella restantes	24
18.5. Recomendaciones operativas	24
19. Predicción de geometría (pestaña Predicción)	25
19.1. Inputs — describir el recinto	25
19.2. Apretar “Predecir” — qué pasa internamente	26
19.3. Las cards — 5 secciones agrupadas	26
19.4. Pesos condicionales por uso	26
19.5. Aplicar un candidato	27
19.6. Evaluar tu diseño actual	27

1. Introducción

Prototipo 1 es una herramienta de escritorio para diseñar recintos acústicos en tres dimensiones, visualizarlos interactivamente y calcular su comportamiento modal usando el Método de Elementos Finitos (FEM).

¿Qué se puede hacer con este software?

- Modelar recintos con geometría arbitraria: polígonos de N lados, techos planos, de arco, a dos aguas o inclinados.
- Visualizar los **modos acústicos** del recinto en 3D (nube de puntos coloreada) y en cortes 2D (mapas de calor).
- Posicionar **fuentes** y un **receptor**, configurar las fuentes por su sensibilidad en dB (como en una ficha técnica).
- Asignar **materiales acústicos** (hormigón, yeso, alfombra, etc.) a piso, techo y paredes, y calcular el RT60 y el amortiguamiento modal resultante.
- Obtener la **FRF** (Respuesta en Frecuencia) en el receptor.
- **Escuchar** cómo afecta la sala al sonido usando ruido rosa filtrado con la FRF.

Requisitos del sistema

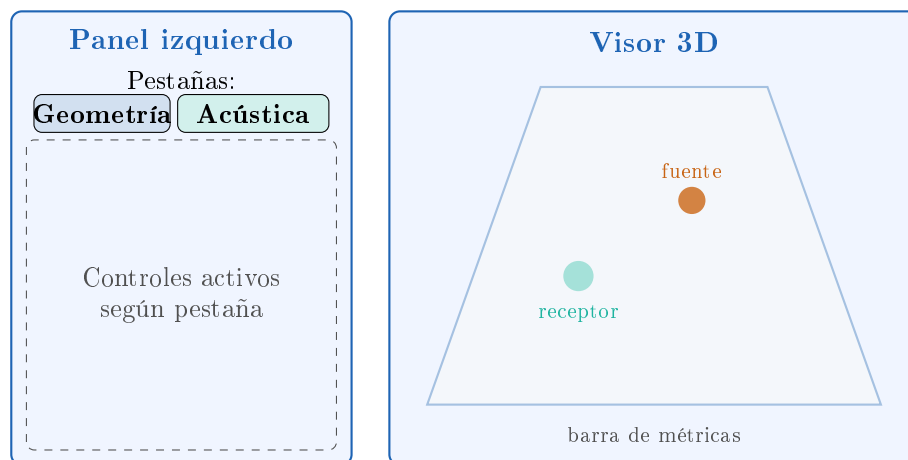
Sistema operativo	Windows 10 / 11 (64 bits)
Python	3.12 (Anaconda)
Dependencias extra	<code>pip install pyqtgraph PyOpenGL matplotlib</code>
Audio	<code>winsound</code> (incluido en Python) + <code>scipy</code>

2. Inicio rápido

1. Abrir el archivo `run.bat` (doble clic). La aplicación arranca con un recinto rectangular de $6 \times 8 \times 3$ m.
2. En la pestaña «**Geometría**», ajustar los sliders de dimensiones y forma del recinto.
3. Cambiar a la pestaña «**Acústica**». Colocar una fuente con `Ctrl` + clic derecho en el visor 3D.
4. Presionar «**Calcular modos (FEM)**».
5. Presionar «**Actualizar campo 3D**» (o `Enter`) para visualizar la distribución de presión modal.
6. Presionar «**Calcular FRF**» para ver la respuesta en frecuencia y «**[Audio] Escuchar**» para oír la sala.

3. Interfaz general

La ventana principal se divide en dos áreas:



4. Diseño del recinto (pestaña Geometría)

4.1 Parámetros básicos

Parámetro	Rango	Descripción
Ancho / Largo / Alto	0.5 – 50 m	Dimensiones del paralelepípedo base
Nº de lados	3 – 32	Polígono de la planta
Afinamiento	0 – 1	Estrechamiento del techo respecto a la planta
Giro	0° – 360°	Rotación del techo sobre su eje vertical

Consejo

Doble clic sobre el valor numérico de cualquier slider para ingresar un valor exacto con el teclado. Doble clic sobre el slider mismo lo resetea a cero.

4.2 Tipos de techo

Tipo	Descripción
Plano	Techo horizontal (default)
Arco	Bóveda de cañón; el slider "Altura de arco" controla la curvatura
Dos aguas	Techo con caballete; "Offset del caballete" lo descentra
Inclinado	Un solo plano inclinado (shed)

4.3 Inclinación de paredes

Con **Clic derecho** + arrastrar sobre una pared en el visor 3D se inclina esa pared de forma interactiva.

4.4 Polígono personalizado

El botón «**Dibujar forma**» abre un editor de polígono 2D:

- **Clic izquierdo** sobre una arista → inserta un vértice.
- **Clic derecho** sobre un vértice → lo elimina.
- Selector de grilla: ajusta el paso de cuadrícula (0.25 m – 5 m).

4.5 Modos de visualización

Modo	Descripción
Aristas	Mesh traslúcido + aristas visibles (default)
Externa	Mesh opaco, sin aristas
Contorno	Solo aristas

5. Controles del visor 3D

5.1 Navegación

Acción	Efecto
Botón central + arrastrar	Órbita libre
Shift + botón central + arrastrar	Órbita horizontal (yaw)
Botón derecho + arrastrar	Paneo
Rueda del mouse	Zoom
0	Reset a vista isométrica

5.2 Interacciones acústicas

Acción	Efecto
Ctrl + clic derecho	Coloca una fuente acústica en el punto del piso bajo el cursor (sensibilidad default: 90 dB).
Shift + clic izquierdo + arrastrar	Mueve la fuente o el receptor más cercano al cursor. La fuente se desplaza en su plano horizontal (z constante). El receptor ídem.
Doble clic sobre una fuente	Abre el diálogo de edición de esa fuente.

6. Módulo acústico (pestaña Acústica)

6.1 Fuentes omnidireccionales

Cada fuente es un monopolo acústico puntual. Se puede colocar:

- Con **Ctrl** + clic derecho en el visor 3D.

- Con el botón «**Añadir**» del panel.

6.1.1 Configuración de la fuente

Al agregar o editar una fuente aparece el siguiente diálogo:

Fuente acústica

Etiqueta: **src_0**

Posición (m): X: 1.00 Y: 1.00 Z: 1.00

Sensibilidad (1W/1m): **90.0 dB SPL**

dB SPL medido a 1 W de potencia eléctrica y 1 m de distancia

→ Q equivalente: $|Q| = 1.045 \times 10^{-3} \text{ m}^3/\text{s}$ (monopolo @ 1000 Hz, 1 W)

Sensibilidad: es el único parámetro de intensidad. Ingresá el valor que figura en la ficha técnica del altavoz: *nivel de presión sonora a 1 W de potencia eléctrica y a 1 m de distancia*, en dB SPL.

Tipo de altavoz	Sensibilidad típica
Subwoofer	85 – 90 dB/W/m
Woofer/full-range	88 – 96 dB/W/m
Tweeter de compresión	100 – 110 dB/W/m
Altavoz de línea (line array)	105 – 115 dB/W/m

El software convierte internamente la sensibilidad a caudal volumétrico Q usando el modelo de monopolo:

$$p_0 = 20 \mu\text{Pa} \cdot 10^{S/20}, \quad |Q| = \frac{p_0 \cdot 4\pi}{\omega_{\text{ref}} \rho_0}$$

donde $\omega_{\text{ref}} = 2\pi \times 1000 \text{ rad/s}$ y $\rho_0 = 1,21 \text{ kg/m}^3$.

Nota

Un incremento de 10 dB en la sensibilidad equivale a multiplicar $|Q|$ por $\sqrt{10} \approx 3,16$, lo cual es coherente con la relación entre potencia y presión acústica.

6.2 Receptor

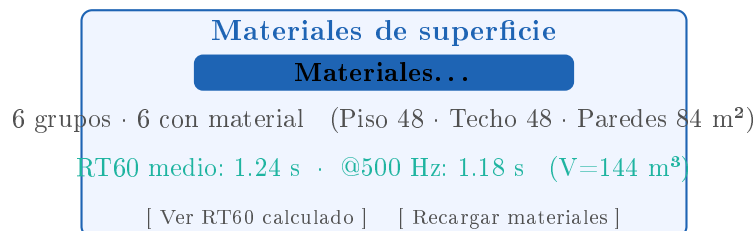
El receptor es el punto donde se evalúa la FRF. Se puede mover:

- Con los spinboxes X/Y/Z del panel.
- Con Shift + arrastrar sobre la cruz cian en el visor 3D.

6.3 Materiales por cara (estilo EASE)

Novedad v2.3. La asignación de materiales ya *no* se hace por zona (piso/techo/paredes) sino **por grupo de caras planares**, igual que en EASE. El recinto se descompone automáticamente

en regiones planares conexas y cada una recibe su propio material. El panel de Acústica muestra un único botón «**Materiales...**» en lugar de los tres combos antiguos:



Cómo abre el diálogo

Apretar «**Materiales...**» abre una ventana modal con una tabla. Una fila por grupo de caras. Columnas: indicador de color, etiqueta automática, número de caras, área en m², categoría (Piso/Techo/Pared/Inclinada) y un combo con el material a aplicar:

	Grupo	Caras	Área (m ²)	Categ.	Material
■	Piso	2	48,00	Piso	Madera ▼
■	Techo	2	48,00	Techo	Yeso ▼
■	Pared 1 (−X (W))	2	24,00	Pared	Hormigón ▼
■	Pared 2 (+X (E))	2	24,00	Pared	Hormigón ▼
■	Pared 3 (+Y (N))	2	18,00	Pared	Alfombra ▼
■	Pared 4 (−Y (S))	2	18,00	Pared	Vidrio ▼

Cómo se detectan los grupos

El agrupador (`face_materials.group_faces_by_planar_region`) hace dos pasos:

1. **Cluster greedy por normal de la cara:** todas las caras cuya normal queda dentro de $\pm 15^\circ$ de un mismo eje quedan en el mismo cluster.
2. **Componentes conexas por cluster:** dentro de cada cluster se separan los grupos no conexos (dos paredes paralelas son dos grupos distintos).

Cada grupo recibe automáticamente una etiqueta legible:

- **Piso / Techo** si la normal apunta hacia abajo/arriba ($|n_z| > 0,85$).
- **Pared N** (con la dirección cardinal aproximada: +X, NE, +Y, NW, −X, SW, −Y, SE) para paredes verticales.
- **Cara inclinada N** para superficies oblicuas (tribunas, plafones inclinados).

Sala $6 \times 8 \times 3$ m: 6 grupos. Pentágono regular: 7. Un auditorio CAD importado puede tener decenas de grupos.

Persistencia automática

- Mientras la app esté abierta, las asignaciones quedan en memoria: cerrar el diálogo y abrirlo de nuevo muestra exactamente las mismas selecciones (no se borran al salir).

- Al **guardar** el `.room`, las asignaciones se serializan en `acoustic.face_materials.assignments` como `{signature: material_name}`, donde `signature` es un hash estable de (normal, centroide, área) redondeados — sobrevive recompilaciones del agrupador y cambios menores de geometría.
- Al abrir un `.room` viejo (v2 o v3) sin asignaciones, todos los grupos arrancan con un material por defecto rígido ($\alpha \approx 0,03$).

Botones de acción rápida

- «**Asignar a todos...**» aplica el material elegido a *todos* los grupos a la vez — útil como punto de partida.
- «**Preset piso/techo/paredes...**» replica el esquema clásico: pide tres materiales y los aplica automáticamente según la categoría de cada grupo.
- «**Recalcular RT60**» fuerza el cómputo de RT60 con la asignación actual sin cerrar el diálogo.

Materiales incluidos:

Material	α a 500 Hz	Uso típico
Hormigón visto	0.02	Muros crudos
Yeso pintado	0.03	Paredes y techos revocados
Ladrillo visto	0.04	Mampostería sin revocar
Madera dura	0.05	Pisos y paneles
Vidrio	0.03	Ventanas y mamparas
Alfombra fina	0.20	Alfombra de pelo corto
Alfombra gruesa	0.40	Alfombra de lana con subpiso
Panel acústico	0.70	Espuma de melamina / lana mineral

Para agregar materiales propios (v2.17): el botón «**Cargar tu material...**» del diálogo Materiales muestra un cuadro con la sintaxis esperada y abre un selector de archivo. El `.json` elegido se **valida** (nombre + absorción por banda), se **copia** a la carpeta `materials/` sin pisar los del catálogo, y la biblioteca se **recarga en el sitio** (`MaterialLibrary.reload`) para que aparezca en todo el programa sin reiniciar. (El camino manual sigue vigente: copiar el `.json` a `materials/` y presionar «**Recargar materiales**».)

Formato del JSON:

```
[{
  "name":          "Mi material",
  "category":      "Porosos",
  "absorption_coef": [0.05, 0.10, 0.20, 0.40, 0.60, 0.70, 0.75, 0.75],
  "scatter_coef":   [0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.35, 0.35]
}]
```

Los 8 valores corresponden a las bandas de octava: 63 / 125 / 250 / 500 / 1000 / 2000 / 4000 / 8000 Hz.

El botón «**Ver RT60 calculado**» abre un gráfico de la curva RT60(f) resultante (fórmula de Sabine), con exportación a PNG, SVG, CSV, TXT y PDF.

6.4 Parches de absorción sub-cara (v2.17)

Además de asignar *un material por cara*, el botón «**Parches de absorción...**» permite dibujar una **región dentro de una cara** con su propio material.

Qué hace físicamente

Un parche **no es física nueva**: le da **resolución sub-cara** al amortiguamiento modal selectivo (criterio A36). Como las formas modales ϕ_n se calculan con paredes rígidas, un parche **no cambia la forma del modo ni el heatmap**; su α entra por dos vías:

1. el **RT60 de Sabine**: le resta área al material anfitrión y aporta la suya ($A = \sum_i \alpha_i S_i$);
2. el ξ_n **por modo**, pesado por la presión modal ϕ_n^2 *sobre la región del parche*:

$$\alpha_{\text{ef}}(n) = \frac{\sum_p \alpha(p) \phi_n(p)^2 dA_p}{\sum_p \phi_n(p)^2 dA_p}$$

Efecto: un modo cuyo antinodo cae sobre el parche se amortigua más; uno con un nodo ahí casi no lo ve. Lo observable es sobre $\xi_n \rightarrow RT \rightarrow \text{FRF}$.

Editor 2D

Se elige la cara de una lista (alcance v1: caras perpendiculares a un eje) y se dibuja sobre su plano local:

- **Modo Rectángulo**: mantener el botón izquierdo y arrastrar.
- **Modo Polígono**: click izquierdo por vértice (convexo o no); se cierra clickeando cerca del primer punto, con `[Enter]` o doble click; **botón derecho** / `[Esc]` deshace el último vértice o cancela.
- **Rueda del mouse** = zoom sobre la grilla, centrado en el cursor.
- Los parches **no pueden solaparse**: si el candidato pisaría a otro se dibuja en rojo y no se agrega (se permite adyacencia por arista).

Los parches se pintan sobre la cara en el visor 3D con el color de su material y se guardan en el `.room` (v8). En el diálogo «**Materiales...**» aparecen listados debajo de las caras (**Parche (rect/polígono) en cara**), con su combo de material editable, y al pasar el cursor por la fila se resaltan en el 3D.

Cuadratura fina (nota importante)

Sin parches, la absorción se integra como siempre (**baseline intacto**: los `.room` sin parches no cambian ni un dígito). Al activar el **primer parche**, ξ_n se recalcula con **cuadratura fina** (tesela la cara en muchos puntos, más preciso que la malla de render gruesa, que usa 1 punto por triángulo). Los números de RT/FRF **pueden moverse** respecto de la malla gruesa: es mayor precisión, no un error. Con material uniforme reduce **exacto** a A36.

Núcleo: `absorption_patch.py` (`compute_xi_per_mode_with_patches`, `sabine_rt60_with_patches`); editor en `patch_dialog.py`; bench `bench_absorption_patch.py`.

7. Cálculo de modos FEM

7.1 Parámetros

Parámetro	Descripción
Nº de modos	Cantidad de modos a calcular (2 – 500). Más modos = más tiempo. Recomendado: 12 para exploración, 30+ para FRF detallada.
Densidad de malla	Nodos por metro para la malla tetraédrica interna (0.5 – 10 m ⁻¹). A mayor densidad, mayor precisión pero mayor tiempo.

Atención

La **densidad de malla** controla la precisión del cálculo FEM, no la cantidad de puntos visibles en la nube 3D. Para más puntos de visualización, usar el spinner «**Resolución campo 3D**».

7.2 Procedimiento

1. Verificar que hay al menos una fuente posicionada dentro del recinto.
2. Presionar «**Calcular modos (FEM)**».
3. Esperar (de segundos a 1–2 minutos según tamaño y densidad).
4. El panel muestra: número de nodos/tetraedros, volumen aproximado y frecuencia máxima de validez del modelo.

La **frecuencia máxima de validez** es:

$$f_{\max} = \frac{c}{6 \cdot h_{\max}}, \quad c = 343 \text{ m/s}$$

Los modos calculados por encima de esta frecuencia tienen errores significativos.

El peor tet define la validez de la malla

La validez la fija el **tetraedro más grande** ($h_{\max}$), no el promedio. **Voxel**: celda uniforme, $h_{\max} = 1/\text{npm}$ exacto. **Gmsh**: recibe `MeshSizeMax =  $h_{\text{target}}$` pero el peor tet sale $h_{\max} \approx 1,5 h_{\text{target}}$ (medido 1,42–1,51). Por eso, en modo *Automático*, el auto-tuner le pide a gmsh una malla 1,5× más fina ($h_{\text{target}} = c/(6 \cdot 1,5 \cdot f_S)$), constante `_GMSH_HMAX_OVER_HTARGET` en `mesh_router.py`) para que la validez *real* alcance f_S y no se quede en $f_S/1,5$. Consecuencia: con gmsh el badge “válido hasta” puede dar un poco **por encima** de f_S (el 1,5 es el peor caso; si los tets salen mejores, valida más alto) — correcto y del lado seguro. Lo que nunca debe pasar es validar *por debajo* de f_S .

Frecuencia de Schroeder

Presionar «**Calcular f_Schroeder**» para ver cuál es la frecuencia límite del régimen modal: por debajo de ella los modos son discretos (FEM es exacto); por encima el campo es estadístico.

8. Visualización del campo acústico

8.1 Nube de puntos 3D

Luego de calcular los modos, presionar «**Actualizar campo 3D**» (o estando en la pestaña Acústica).

El selector «**Campo**» cambia lo que se muestra:

Opción	Descripción
Forma modal	Muestra la forma de vibración $\varphi_n(\mathbf{x})$ del modo seleccionado. No depende de la posición de la fuente. Colormap: azul = fase negativa, blanco = nodo, rojo = fase positiva.
Presión p	Muestra la amplitud $ p(\mathbf{x}, f_n) $ generada por las fuentes a la frecuencia del modo seleccionado. Cambia al mover la fuente (actualización automática a los 350 ms). Colormap: azul = mínimo, rojo = máximo.

Nota

En la frecuencia de resonancia, la distribución de $|p|$ se parece mucho a la forma modal porque el modo dominante escala el patrón. La diferencia se nota al mover la fuente a un *nodo* del modo: $\varphi_n(\mathbf{x}_s) = 0$, la excitación de ese modo cae a cero y el patrón cambia completamente.

El spinner «**Resolución campo 3D**» controla la densidad de puntos:

Valor	Puntos aprox. (sala $6 \times 8 \times 3$)
20 (default)	600 – 1500 pts (rápido)
40	5000 – 8000 pts (medio)
60	12000+ pts (detallado, lento)

8.2 Plano de corte 2D — flujo interactivo

El plano de corte permite ver la distribución modal o de presión en una sección transversal del recinto.

1. En «**Plano de corte**» elegir la orientación: **XY** ($z = \text{cte}$), **XZ** ($y = \text{cte}$) o **YZ** ($x = \text{cte}$).
2. Presionar « $\oplus$ **Activar plano interactivo**».
3. Mover el cursor sobre el recinto 3D $\rightarrow$ aparece un plano celeste translúcido que sigue el cursor.
4. $\rightarrow$ confirma la posición y **abre automáticamente el mapa de calor 2D**.
5. $\rightarrow$ cancela sin confirmar.

Alternativa: tipear el valor directamente en el spinner de posición y presionar «**Ver mapa de calor 2D**».

8.2.1 Mapa de calor 2D

Se abre una ventana **no modal** (puede quedar abierta mientras se sigue trabajando). Si se confirma un nuevo plano, la ventana se actualiza sin abrir una nueva.

Modo « Forma modal »	Colormap divergente (azul–blanco–rojo). La leyenda muestra "Amplitud modal (normalizada)".
Modo « Presión p »	Colormap inferno (negro–naranja–blanco). La leyenda muestra el nivel en dB SPL (re 20 µPa) .

La zona fuera del recinto aparece en gris. Se puede exportar a **PNG, SVG, PDF** (imagen) o **CSV / TXT** (valores numéricos del corte en formato largo: **x**; **y**; **amplitud_modal** en forma modal, o **x**; **y**; **presion_pa**; **spl_db** en presión).

9. Respuesta en Frecuencia (FRF)

9.1 Cálculo

Presionar «**Calcular FRF**» en el grupo "FRF (Respuesta en frecuencia)". Parámetros:

Parámetro	Descripción
f mín / f máx	Rango de frecuencias del gráfico.
Nº de puntos	Resolución en frecuencia (10 – 1000). Más puntos = curva más suave pero más tiempo de cálculo.

9.2 Gráfico de FRF

El eje Y muestra **nivel de presión en dB SPL** (referencia 20 µPa) asumiendo una fuente de 1 W de potencia eléctrica.

Las **líneas naranjas discontinuas** marcan las frecuencias de los modos calculados por FEM.

Consejo

Doble clic en el gráfico → resetea el zoom al rango completo. La barra de herramientas permite hacer zoom, panear y guardar la figura.

Exportar (v2.8): imagen (PNG / SVG / PDF) o datos crudos (CSV / TXT). Los archivos de datos llevan columnas **freq_hz**; **spl_db**; **abs_H_pa**; **phase_deg** para reabrir en Excel, Python o MATLAB. CSV usa ; con coma decimal (Excel-es); TXT usa tabulador con punto decimal.

9.3 Escucha — ruido rosa filtrado por la sala

El botón **Escuchar** permite oír cómo afecta la sala al sonido:

1. Se generan 4 segundos de **ruido rosa** (densidad espectral $\propto 1/f$).
2. Se filtra con la FRF calculada usando convolución en frecuencia.
3. Se aplica **boost de ganancia +6 dB equivalente** con soft-clipping tanh (v2.5).

4. Se ponen **fade-in** 10 ms y **fade-out** 50 ms lineales para eliminar el *pop* al iniciar y al terminar.
5. Se anexan 100 ms de **silencio** al final del WAV para que el buffer del DAC termine en cero (anti-chasquido extra).
6. Se reproduce por los parlantes del sistema.

El audio resultante es un archivo WAV de: **16 bits / 44 100 Hz / estéreo** (L = R, campo mono).

El botón «**Detener**» interrumpe la reproducción.

Nota

El ruido rosa solo tiene coloración de sala en el rango $[f_{\text{mín}}, f_{\text{máx}}]$ calculado. Para incluir más rango de frecuencias, ampliar esos parámetros antes de calcular la FRF.

10. RT60 y materiales — comparativa de métodos

El botón «**Ver RT60 calculado**» abre un diálogo **multi-curva** donde se pueden combinar dos métodos de predicción (Sabine y Eyring), agregar y quitar curvas para comparar, y exportar a **PNG / SVG / PDF** (imagen) o **CSV / TXT** (valores numéricos: una columna **banda_hz** y una por cada curva activa, ej. **Sabine RT60_s**).

Nota

En la v2.5 se simplificó el diálogo: se removieron de la UI el método Fitzroy y las métricas T20 / T30. El código Fitzroy permanece en `face_materials.compute_fitzroy_rt60_per_face` por si quiero re-habilitarlo; $T20 = T30 = T60$ en predicciones teóricas (decaimiento exponencial puro), por lo que sólo se grafica T60.

10.1 Métodos de predicción

Sabine (clásico):

$$A(f) = \sum_g \alpha_g(f) S_g, \quad \text{RT}_{60}(f) = \frac{0,161 \cdot V}{A(f)}$$

Asume $\alpha \ll 1$. Sobreestima RT60 para α alta (no llega a cero en sala anecoica).

Norris–Eyring (corrige absorción alta):

$$\bar{\alpha}(f) = \frac{A(f)}{S_{\text{total}}}, \quad \text{RT}_{60}(f) = \frac{0,161 \cdot V}{-S_{\text{total}} \ln(1 - \bar{\alpha}(f))}$$

Cuando $\bar{\alpha} \rightarrow 1$, $-\ln(1 - \bar{\alpha}) \rightarrow \infty$ y $\text{RT}_{60} \rightarrow 0$ (sala anecoica), que es lo físicamente correcto. En el límite $\bar{\alpha} \ll 1$, $-\ln(1 - \bar{\alpha}) \approx \bar{\alpha}$ y la fórmula colapsa a Sabine.

10.2 Diálogo comparativo

Lado izquierdo: gráfico matplotlib con todas las curvas activas (eje X logarítmico de 63 a 8000 Hz; eje Y en segundos).

Lado derecho: lista de curvas con **casilla de visibilidad por curva** (ocultar sin borrar) y botón «**Quitar curva seleccionada**». Más abajo, un combo para elegir método (*Sabine* o *Eyring*) y un botón «**Agregar curva con la asignación actual**». Botón final «**Borrar todas las curvas**».

Cada curva guarda un **snapshot numérico** de la materialización en el momento de agregarla. Esto permite que el usuario cambie los materiales en la ventana «**Materiales**» entre clics y compare variantes (la curva nueva se llama «**Sabine RT60 #2**», etc.). Las curvas viejas no se modifican al cambiar materiales.

10.3 Código de colores

Visual	Significado
Azul (#1f77b4), sólida con círculos	Sabine RT60
Rojo (#d62728), sólida con círculos	Eyring RT60

10.4 Amortiguamiento modal

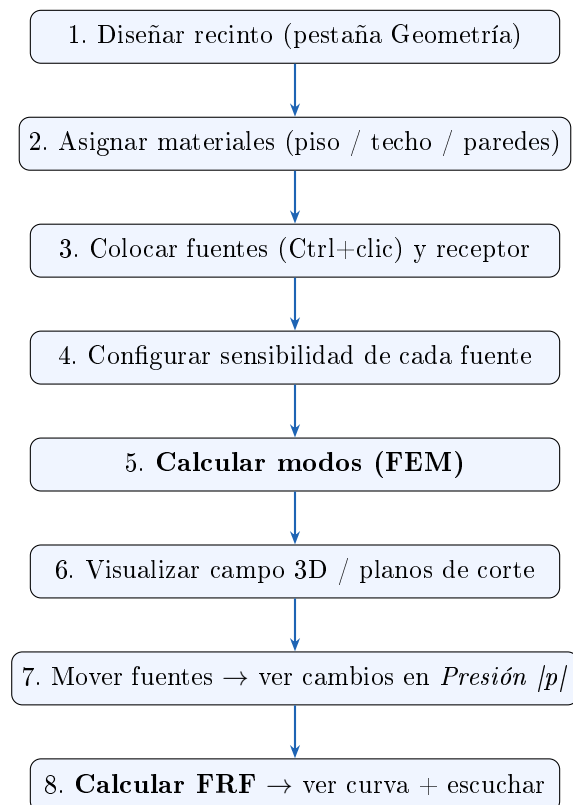
El amortiguamiento ξ_n usado por el FEM para anchar los picos de la FRF se calcula a partir del RT60 de **Sabine** (por compatibilidad con los valores históricos del solver):

$$\xi_n = \frac{1,1}{f_n \cdot \text{RT}_{60}(f_n)}$$

Esto hace que los picos de la FRF sean más o menos anchos según la absorción del recinto: materiales muy absorbentes $\rightarrow$ RT60 corto $\rightarrow$ ξ_n grande $\rightarrow$ picos anchos.

Si querés que el amortiguamiento modal use Eyring en vez de Sabine, basta con cambiar la llamada a `compute_sabine_rt60_per_face` en `acoustic_panel._compute_xi_from_materials` por `compute_eyring_rt60_per_face`.

11. Flujo de trabajo completo



12. Referencia rápida de atajos

Atajo	Acción
Ctrl + Z	Deshacer
Ctrl + Y	Rehacer
Ctrl + S	Guardar recinto
Ctrl + Shift + S	Guardar como
Ctrl + O	Abrir recinto
O	Reset cámara vista isométrica
Enter (pestaña Acústica)	Actualizar campo 3D
Ctrl + clic derecho (visor)	Colocar fuente en el piso
Shift + clic izquierdo + arrastrar	Mover fuente/receptor
Doble clic en fuente	Editar fuente

13. Solución de problemas frecuentes

Problema	Solución
La aplicación no arranca	Verificar que se usa <code>run.bat</code> (no <code>python main.py</code> directamente). El bat usa la ruta de Anaconda explícitamente.
La nube 3D no aparece	Verificar que se calcularon los modos primero. Verificar que hay al menos una fuente en modo "Presión $ p $ ".
Las fuentes no se ven en el visor	Cambiar a modo de visualización Aristas (mesh traslúcido). En modo <code>.Externa</code> (opaco) las fuentes interiores quedan tapadas por las paredes.
"Forma modalz" "Presión $ p $ " lucen igual	Es física: en resonancia, $ p \propto \varphi_n $. Mover la fuente a un nodo del modo para ver la diferencia.
El audio no suena	Verificar que los parlantes del sistema están activos y el volumen de Windows no está en cero. El audio usa la API nativa de Windows.
FRF tarda mucho	Reducir el número de modos o el número de puntos de la FRF. Para exploración rápida: 8 modos, 200 puntos.
Geometría cambia → modos se borran	Comportamiento esperado: al cambiar el recinto la malla FEM anterior ya no corresponde. Recalcular modos con « Calcular modos (FEM) ».

14. Conceptos físicos clave

14.1 Modos acústicos

Los modos son las frecuencias naturales del recinto: a esas frecuencias la presión se distribuye de forma estacionaria (patrón fijo de nodos y antinodos). La forma modal $\varphi_n(\mathbf{x})$ muestra dónde hay máxima presión (antinodos) y dónde la presión es cero (nodos).

14.2 Sensibilidad vs. caudal volumétrico Q

$$\begin{aligned}
 S &= \text{sensibilidad en dB SPL @ 1W/1m} \\
 p_0 &= 20 \mu\text{Pa} \cdot 10^{S/20} \quad (\text{presión a 1W/1m}) \\
 |Q| &= \frac{p_0 \cdot 4\pi}{\omega_{\text{ref}} \rho_0} \quad \text{con } f_{\text{ref}} = 1 \text{ kHz}
 \end{aligned}$$

Para una fuente de 90 dB/W/m: $|Q| \approx 1,05 \times 10^{-3} \text{ m}^3/\text{s}$.

14.3 Amortiguamiento modal y RT60

El amortiguamiento ξ_n es la fracción de energía que se pierde por ciclo en el modo n . Un recinto con paredes de hormigón tiene $\xi_n \approx 0,005$ (picos muy agudos en la FRF, RT60 largo). Una sala tratada acústicamente puede llegar a $\xi_n \approx 0,05$ (picos anchos, RT60 corto).

14.4 Frecuencia de Schroeder

Es la frecuencia límite entre el régimen modal (modos individuales) y el campo estadístico (muchos modos superpuestos):

$$f_S = 2000 \sqrt{\frac{RT_{60}}{V}}$$

El FEM es válido y útil por debajo de f_S . Por encima, los modos son tan densos que el campo parece continuo.

15. Importar CAD y motor de mallado

A partir de la versión 2.0, Prototipo 1 puede trabajar con **geometrías arbitrarias importadas desde CAD** (auditorios, salas con curvas complejas, modelos arquitectónicos profesionales) y elige automáticamente el motor de mallado más apropiado para cada caso. La versión 2.1 añade el modo **best-effort** con fallback automático.

15.1 Por qué hay dos motores

El cálculo modal FEM requiere descomponer el volumen del recinto en tetraedros. Hay dos estrategias para hacerlo:

- **Voxel**: rejilla estructurada de cubos partida en tetraedros, filtrando los que caen fuera del recinto. Ideal para recintos *axis-aligned*: paredes verticales, techo plano, aristas paralelas a los ejes X/Y . En ese caso, las celdas voxel coinciden **exactamente** con las paredes y el motor es **exacto sin overhead**.
- **Gmsh** (boundary-fitted): mallador profesional con kernel OpenCASCADE que genera tetraedros que se ajustan **exactamente** a la geometría, por curva que sea. Necesario para recintos con paredes curvas, oblicuas o cualquier malla importada de CAD.

Evidencia experimental — cilindro $R = 4$ m, $H = 4$ m

Motor	Volumen calculado	t_{total}	Split modos 1/2
Gmsh boundary-fitted	200.26 m ³ (err -0,4%)	0.51 s	0.0017 Hz
Voxel forzado	201.39 m ³ (err +0,16%)	2.99 s	0.0275 Hz

Los modos 1 y 2 son **físicamente degenerados** (igual frecuencia por simetría rotacional). Gmsh preserva esa degeneración con un split numérico de 0.0017 Hz; voxel la rompe con 0.0275 Hz. En geometría curva, **gmsh es 6× más rápido y 16× mejor para preservar simetrías físicas**.

15.2 El badge de estado

En la pestaña **Acústica**, dentro del grupo *FEM modal*, hay un **badge coloreado** que indica qué motor se va a usar y por qué:

Color	Texto	Significado
verde	voxel · exacto	Recinto axis-aligned: voxel coincide con la frontera. Sin error de escalera. Óptimo .
azul	gmsh · boundary-fitted	Geometría curva o CAD importado: gmsh ajusta la malla a las paredes exactas. Óptimo .
amarillo	voxel · escalera voxel · fallback	Geometría curva con voxel: error sistemático. Si dice fallback , gmsh fue intentado y falló. Tooltip con la razón exacta.
naranja	gmsh · forzado	Forzaste gmsh donde voxel sería exacto. Sin perjuicio.

15.3 Importar un archivo CAD

Tres formas de invocar el importador:

1. Botón «**[+]** Importar CAD...» en el grupo *FEM modal*.
2. Atajo de teclado **Ctrl** + **I**.
3. Arrastrá un **.stl/.obj/.step** sobre el ejecutable (en una futura versión).

Al elegir un archivo, el soft hace tres cosas:

1. **Carga** la malla (**trimesh** para mallas triangulares, **gmsh**+OpenCASCADE para B-rep como STEP/IGES).
2. **Diagnostica** la malla: cuenta vértices, triángulos, calcula el volumen, mide si es *watertight* (cerrada), si tiene caras degeneradas, si hay aristas no-manifold, y **lista todos los huecos**.
3. Si la malla está perfecta → la usa directamente. Si tiene problemas → abre el **diálogo de reparación guiada** (sección 15.4).

15.4 Diálogo de reparación guiada

Cuando el CAD tiene problemas (huecos, T-junctions, vértices duplicados), aparece una ventana con tres zonas: resumen de la malla, lista de huecos con preview 3D y acciones de reparación.

Acciones por hueco:

- **Cerrar hueco (auto)**: triangulación por abanico desde el centroide. Funciona para huecos planos chicos.
- **Soldar a vecinos**: fusiona vértices del borde del hueco con vértices cercanos del resto de la malla (T-junctions, duplicados).
- **Mover vértice**: sub-diálogo donde elegís un vértice del ciclo del hueco y le asignás una nueva posición XYZ exacta.

- **Omitir:** pasa al siguiente hueco sin tocar.

Acciones globales:

- **Reparar TODO automáticamente:** cierra todos los huecos en cadena.
- **Fusionar vértices duplicados:** junta vértices a distancia <0.1 mm. Común en STL.
- **Normalizar (winding/normales):** orienta normales hacia afuera y arregla winding inconsistente.

El preview 3D muestra el hueco actual en rojo grueso, con esferas naranjas en los vértices del ciclo y una esfera amarilla en el centroide.

15.5 Override manual del motor

Junto al badge hay un combo «Motor de mallado» con tres opciones:

- **Automático** (*default*): el router decide según las reglas de la sección 15.8.
- **Voxel:** fuerza voxel.
- **Gmsh:** fuerza gmsh.

El override se persiste por proyecto (campo `acoustic.mesh_engine` en el `.room`) y como default global (en `%APPDATA%\Prototipo1\settings.json`).

Override "Gmsh- fallo de gmsh"

Si forzás **Gmsh** y la malla no es topológicamente válida, el sistema **lanza una excepción explícita** en lugar de caer silenciosamente a voxel. La razón: vos pediste gmsh expresamente, no querés un fallback sin darte cuenta. Para fallback automático, dejá el motor en **Automático**.

15.6 Formatos soportados

Familia	Formatos	Loader interno
Mallas triangulares	STL (ASCII y binario), OBJ, PLY, glTF/GLB, 3MF, COLLADA, OFF, XYZ	trimesh
B-rep paramétrico (CAD profesional)	STEP, STP, IGES, IGS, BREP	gmsh (OpenCASCADE)

Los formatos B-rep se teselan automáticamente al cargar. Un STEP de un auditorio importado desde Revit o SolidWorks llega al motor FEM **sin pérdida geométrica**.

15.7 Persistencia en `.room v3`

El formato `.room` evolucionó de v2 a v3 para guardar:

```
{
  "format": "prototipo1.room",
  "version": 3,
  "params": { ... },           // sliders de geometría
```

```

"acoustic": {
  "mesh_engine": "auto",      // "auto" / "voxel" / "gmsh"
  "h_target": 0.40,          // tamaño de tetraedro para gmsh
  "n_per_meter": 2.5,        // densidad para voxel
  "n_modes": 12,
  "sources": [...],
  "receiver": [3.0, 4.0, 1.5]
},
"external_geometry": {      // solo si hay CAD importado
  "kind": "embedded_mesh",
  "vertices": [[x,y,z], ...],
  "faces":    [[i,j,k], ...]
}
}

```

Importante: el `.room` embebe una copia completa de la malla CAD importada. Eso lo hace **autocontenido y portable**: podés mandar el `.room` por mail sin necesidad de adjuntar el STEP/STL original. Tamaño extra típico: 5–500 KB.

Los archivos `.room` v2 antiguos siguen abriéndose sin problema (los campos nuevos son opcionales).

15.8 Cuándo usa cada motor el router automático

A partir de v2.1 el router opera en modo **best-effort**: para geometrías curvas intenta gmsh primero y, solo si falla por incompatibilidad topológica, cae a voxel reportando la razón.

Geometría	Auto-decisión	Si gmsh OK	Si gmsh falla
Shoebox axis-aligned, techo plano	voxel directo	● voxel exacto	n/a
Pentágono regular techo plano	voxel directo	● voxel exacto	n/a
<code>arch_height > 0</code>	intenta gmsh	● gmsh boundary-fitted	● voxel fallback
<code>twist/taper/pitch ≠ 0</code>	intenta gmsh	● gmsh	● voxel fallback
Polígono custom no axis-aligned	intenta gmsh	● gmsh	● voxel fallback
CAD importado (cualquiera)	intenta gmsh	● gmsh	● voxel fallback

15.9 Workflow recomendado para auditorios complejos

1. **Modelar en CAD externo** (Revit, SketchUp, SolidWorks, Blender, FreeCAD...).
2. Exportar como **STEP** (mejor calidad) o **STL** (universal).
3. En Prototipo 1, `Ctrl` + `I` → seleccionar el archivo.
4. Si hay huecos, reparar con el diálogo guiado.
5. Posicionar fuentes y receptor.

6. Asignar materiales (piso/techo/paredes).
7. «**Calcular modos (FEM)**» — el router elige gmsh automáticamente.
8. Visualizar modos en 3D, hacer FRF, escuchar la sala.
9. `Ctrl` + `S` para guardar el proyecto completo (con CAD embebido).

15.10 El sistema best-effort y el fallback

El router funciona en modo **best-effort**: para cualquier geometría no axis-aligned, intenta gmsh primero. Solo si gmsh detecta una incompatibilidad topológica (T-junctions, normales inconsistentes), cae automáticamente a voxel y muestra un badge amarillo `voxel · fallback` con la razón exacta en el tooltip.

Por qué existe el fallback

Una malla puede ser **visualmente correcta** pero **topológicamente inválida** para un mallador profesional. Ejemplos: T-junctions (dos triángulos comparten arista, un tercero tiene un vértice sobre esa arista que los otros no conocen), aristas no-manifold (3+ triángulos compartiendo una arista), normales inconsistentes.

Para el render OpenGL estos problemas son irrelevantes. Para gmsh, son fatales. La opción anterior era abortar; con best-effort, el sistema sigue funcionando con voxel y avisa qué pasó.

Cómo se ve en la UI cuando hay fallback

Después de presionar «**Calcular modos (FEM)**»:

1. El log de estado muestra:

```
\textcolor{cOrange}{\textbf{!}} Gmsh intentado pero falló (PLC Error: A segment and a face
intersect at point). Cayendo a voxel.
FEM listo (voxel). Malla: 2386 nodos, 9504 tets, V$\approx$177.84 m³.
```

2. El badge cambia de `gmsh · boundary-fitted` a `voxel · fallback`.
3. El tooltip del badge muestra la razón completa del fallo.

Qué hacer cuando ves `voxel · fallback`

1. **Mirá la razón en el tooltip del badge.** Si dice "PLC Error.º "T-junctions", tu malla tiene aristas mal conectadas.
2. **Opción rápida:** aceptás el voxel con error de escalera ($\sim 0,5\text{--}1\%$ en frecuencias, degeneraciones rotas).
3. **Opción profesional:** re-importás el CAD con `Ctrl` + `I` y aplicás reparación guiada.
4. **Opción exhaustiva:** revisás la malla en el CAD original (re-exportar con "merge close vertices" activado en STL).

16. Escala y orientación al importar

Cuando se carga un archivo CAD con **Ctrl** + **I**, antes del diálogo de reparación aparece un **diálogo de escala y orientación**. Resuelve dos problemas frecuentes al importar geometría externa: unidades desconocidas y eje vertical no coincidente.

16.1 Escala — unidades del archivo

Los formatos CAD no llevan información de unidades estandarizada. Un STL puede estar dibujado en milímetros, centímetros, metros o pulgadas. Si el archivo viene en milímetros y el soft lo interpreta como metros, el recinto queda mil veces más grande de lo esperado y se sale completamente de la grilla.

El diálogo detecta automáticamente la unidad probable midiendo la **diagonal del AABB**:

Diagonal del AABB	Sugerencia	Razón
> 5 000 m	÷1 000 (mm → m)	Archivo en milímetros
> 500 m	÷100 (cm → m)	Archivo en centímetros
> 60 m	÷10	Grande para recinto típico
0,5 – 60 m	×1	Rango plausible
< 0,5 m	×100 o ×1 000	Demasiado pequeño

El usuario puede aceptar la sugerencia, elegir un preset (mm/cm/dm/m, pulgadas, pies), ingresar un factor manual con precisión, o usar el botón **«Auto-encajar diagonal a 20 m»** para forzar la diagonal a ese valor sin importar las unidades originales.

16.2 Orientación — eje vertical del archivo

Los formatos CAD también difieren en qué eje consideran **vertical**:

Formato	Convención típica
Prototipo 1, FreeCAD, AutoCAD, STEP, IGES, STL técnico	Z+ up
OBJ, glTF, GLB, COLLADA, FBX, 3MF (Blender, Unity, Unreal)	Y+ up

Si un OBJ Y-up se importa sin reorientar, la pared trasera queda como piso. El combo de orientación permite elegir la convención del archivo y aplica internamente una rotación rígida 3D que lleva el eje seleccionado al Z+ del soft.

Sugerencia automática por extensión

Si el archivo termina en **.obj**, **.gltf**, **.glb**, **.dae**, **.fbx** o **.3mf**, el combo arranca preseleccionado en **Y+ up**. Para el resto (STL, STEP, IGES, BREP, PLY) arranca en **Z+ up**. El usuario siempre puede cambiar el default si la convención del archivo no es la habitual del formato.

Al confirmar el diálogo, se aplica primero la **reorientación** y después la **escala**, de modo que las nuevas dimensiones del preview ya están expresadas en los ejes del soft (X, Y horizontales; Z vertical).

16.3 Tres botones del diálogo de escala (v2.8)

Botón	Qué hace
«Aplicar escala»	Usa el factor + orientación elegidos y continúa la importación.
«No escalar»	Saltea el escalado (factor = 1,0) pero igual continúa la importación . Útil cuando ya sabés que el archivo está en metros.
«Cancelar import»	Aborta toda la importación. Esc y el botón × de la ventana hacen lo mismo.

Cambio en v2.8

Antes “No escalar” cancelaba la importación entera (bug histórico — el botón decía “Cancelar” con texto “No escalar”). Ahora son tres opciones explícitas con semántica clara.

16.4 Optimizaciones del reparador de mallas

Las funciones de reparación en `geom_import.py` se reescribieron para escalar bien con mallas grandes:

- `find_holes` vectorizada con `numpy.unique`. Para mallas de ~ 100 k caras: ~ 50 ms en vez de ~ 3 s del bucle Python.
- `fill_all_holes_auto` en un único pase: detecta todos los huecos, construye los parches en arrays NumPy y materializa el `Trimesh` una sola vez. Para K huecos sobre N triángulos: $O(N + K)$ en vez de $O(N \cdot K)$.
- `snap_hole_vertices` usa `scipy.spatial.cKDTree` para nearest-neighbor en lote. Para mallas con > 10 k vértices: $\sim 100\times$ más rápido.

En la práctica: un cilindro de ~ 4 k triángulos con 50 huecos forzados se repara completo en ~ 350 ms.

17. Indicador de ejes y rotación con eje fijo

En la **esquina inferior derecha** del visor 3D aparece un pequeño rectángulo con tres cuadrados etiquetados X, Y, Z. Es el **indicador de ejes** y sirve para restringir la rotación de la cámara a un eje mundial específico.

17.1 Fijar un eje

Dos formas equivalentes:

1. **Click izquierdo** sobre el cuadrado X, Y o Z.
2. **Atajo de teclado:** `Ctrl` + `Shift` + `Alt` + `X` (o `Y`, o `Z`).

Ambos comportamientos son **toggle**: aplicar de nuevo el mismo libera el eje. Cuando un eje queda fijo, su cuadrado se ilumina con un color asociado a la flecha del eje en la escena:

Eje	Color del cuadrado al activarse
X	rojo
Y	azul
Z	verde

17.2 Comportamiento de la rueda del mouse con eje fijo

- **Sin eje fijo** (comportamiento original): rueda presionada + arrastrar = orbit libre (cambia azimuth y elevation).
- **Con eje fijo**: rueda presionada + arrastrar **horizontalmente** = el recinto rota únicamente alrededor del eje mundial fijado. El componente vertical del mouse se ignora para que el control sea predecible.

Internamente: el visor calcula la posición actual de la cámara en coordenadas esféricas, aplica una matriz de rotación 3D alrededor del eje fijo y recalcula **azimuth** / **elevation** / **distance** para los nuevos valores.

Cuándo usar esta función

- Inspeccionar el **perfil lateral** del recinto: fijar Y o X y rotar horizontalmente preserva la altura (Z).
- Mantener una **vista superior** mientras se rota la planta: fijar Z, sólo cambia el azimuth.
- Documentar la geometría desde un eje específico para una figura del informe.
- Comparar **perfiles simétricos** del recinto entre rotaciones controladas.

18. Rendimiento y benchmarks

A partir de la versión 2.3, el repositorio incluye `benchmark_v2.py`, una suite *headless* (sin GUI) que mide los principales caminos de la app y deja un reporte completo en `BENCHMARK_RESULTS.md`.

18.1 Cómo correr los benchmarks

```
%USERPROFILE%\anaconda3\python.exe" benchmark_v2.py
```

El script demora 2–5 minutos (depende del CPU). Mientras corre imprime cada bloque en consola y al final escribe `BENCHMARK_RESULTS.md` en la raíz del proyecto.

18.2 Qué mide

Bloque	Qué evalúa
B1	Tiempo de FEM completo (mallado + K/M + Lanczos).
B2	Campo 3D a resoluciones 20, 30, 40, 50, 60, 70.
B3	Comparativa loop Python (antes) vs KDTree (ahora).
B4	Agrupación de caras por región planar.
B5	Importación CAD descompuesta en fases.
B6	RT60 con asignación por grupo.
B7	Memoria del KDTree del FieldEvaluator.

Cada test corre tres veces y reporta mediana + min/max.

18.3 Resultados de referencia

Los números varían con el equipo, pero estos son representativos en una CPU de 16 hilos con 64 GB de RAM:

- **Campo 3D a resolución 50** (62 500 puntos): ~ 280 ms. Antes de la optimización: 15–25 s.
- **Campo 3D a resolución 70** (170 000 puntos): ~ 740 ms.
- **Comparativa loop vs KDTree**: 50–170 $\times$ más rápido, diferencia numérica $< 10^{-15}$ (redondeo IEEE-754).
- **Agrupación de caras**: $< 2,5$ ms para cualquier recinto paramétrico.
- **Importación CAD**: 20 ms (5 k tris) $\rightarrow$ 1,2 s (327 k tris).
- **Cálculo de RT60 por grupo**: < 1 ms.

18.4 Cuellos de botella restantes

Una vez vectorizada la evaluación del campo (el cuello histórico), el tiempo dominante del cálculo FEM está en el **mallado volumétrico** (`acoustic_mesh.build_volume_mesh`):

Sala	npm	tets	mallado
$6 \times 8 \times 3$	2,5	14 400	1,2 s
$6 \times 8 \times 3$	3,5	35 280	3,0 s
$10 \times 10 \times 4$	2,5	28 140	3,4 s

Por eso `BENCHMARK_RESULTS.md` recomienda mantener **npm** $\leq 3,0$ para diseño cotidiano. Subir a 3,5–4,0 sólo cuando se necesita precisión arriba de 300 Hz y se acepte esperar 5–10 s por cada recálculo de modos.

18.5 Recomendaciones operativas

1. **Resolución del campo 3D**: **40–50** para exploración ($< 0,3$ s); subir a **70** solo para figuras finales ($\sim 0,7$ s).

2. **Densidad de malla FEM** (`n_per_meter`): **2,5** para el día a día; **3,0–3,5** cuando se requiera precisión a alta frecuencia.
3. **Importación de CADs grandes**: archivos de > 100 k triángulos van a tardar 0,5–1,5 s por la fase de *diagnose*. El nuevo `QProgressDialog` muestra qué está pasando y permite cancelar.
4. **Materiales por cara**: el diálogo se puede abrir y cerrar libremente; cambiar asignaciones recomputa RT60 en menos de 1 ms (UI totalmente reactiva).

19. Predicción de geometría (pestaña Predicción)

A partir de **v2.6** hay una tercera pestaña, **Predicción**, que invierte el flujo de trabajo: en vez de diseñar una sala y después medir si es buena, le decís al soft **qué necesitás** (uso, capacidad, restricciones del local) y te propone **3 alternativas de dimensiones** que cumplen tus objetivos, scoreadas por **13 criterios** acústicos y prácticos.

19.1 Inputs — describir el recinto

El panel tiene 4 grupos de controles:

Uso del recinto. Combo con 8 presets (Sala de conferencias, Aula, Estudio control room, Estudio live room, Home theater, Sala de música cámara, Sala sinfónica, Sala polivalente). Cada uno tiene RT60 objetivo y V/persona típicos basados en Beranek / ANSI / Long. Combo dependiente de *Programa* (voz hablada / amplificada / música / cine) y slider de *Prioridad* Inteligibilidad $\leftrightarrow$ Envoltura.

Audiencia. Capacidad (personas), m^2 por persona (default *auto*: 0,80 para voz, 1,00 para música, 1,20 para cine), área total calculada.

Restricciones del local (opcional). Caps de ancho/largo máx; **override de altura** (v2.8: por defecto los candidatos tienen muros entre 2,5 y 4,0 m por motivo constructivo — ver nota abajo; activar para forzar otro techo máximo, mayor o menor); combo de paredes paralelas *Permitir* / *Evitar* (este último agrega `taper=0.15`); combo de forma de techo (Plano / Inclinado / Abovedado).

Por qué 2,5–4 m por defecto (v2.8)

Una jornada típica de albañilería coloca ~ 13 hiladas de ladrillo (≈ 10 cm cada una), o sea 1,3 m/día. Un muro de 5 m son 3 jornadas + apuntalamiento + riesgo de derrumbe. Si tu volumen objetivo daría una H más alta (ej. 200 personas $\times 6 \text{ m}^3/\text{p} \rightarrow V = 1200 \text{ m}^3$, Bolt sugiere $H \approx 7,7$ m), el clamp lleva H a 4,0 m y **reescala W y L preservando su proporción $W:L$** para mantener el volumen. Si efectivamente querés un techo de 10 m (sala sinfónica, iglesia), activá «**Override altura**» y subí el spinbox.

Objetivos acústicos (auto-llenado por uso). RT60 @ 500 Hz y V por persona; se rellenan del preset pero se pueden editar.

19.2 Apretar “Predecir” — qué pasa internamente

1. Volumen objetivo: $V = \text{capacidad} \times V/\text{persona}$.
2. Se generan **3 candidatos** con ratios clásicos (Bolt 1,9:1,4:1, Bonello 1,59:1,26:1, Loudon 2,33:1,6:1), escalados para llegar a V y respetando las restricciones de planta/altura. **Clamp constructivo** (v2.8): si la H del ratio textbook cae fuera de $[2,5; 4,0]$ m (o del override del usuario), se recorta H y se reescalan W y L preservando su proporción para conservar V . Esto rompe deliberadamente el ratio $L:W:H$ textbook a favor de una sala construible; la proporción $L:W$ (la que más afecta la distribución modal lateral) se mantiene intacta.
3. Sobre cada candidato se corre un **FEM ligero en paralelo** (`ThreadPoolExecutor` con 3 workers, malla coarse, 40 modos) que cubre hasta ~ 125 Hz.
4. Se calculan los **13 sub-scores** y un total ponderado por grupo.
5. Si los 3 candidatos quedan dentro de ± 5 puntos, se agrega una **4ª card “Control Negativo (Cubo 1:1:1)”** con border rojo punteado.

Tiempo total: ~ 4 segundos con `QProgressDialog`.

19.3 Las cards — 5 secciones agrupadas

Cada card se organiza en 5 bloques (algunos se ocultan según el uso):

- **MODAL** (siempre): RT60 feasibility (qué α requiere la sala), Bolt-spacing por bins de 5 Hz (buenos / grumos / huecos), Modal Q audibility (modos con $Q > 30$, audibles individualmente), Schroeder coverage.
- **VOZ** (oculto para usos de música pura): STI por Bradley, %Alcons por Peutz, distancia crítica vs receptor típico.
- **MÚSICA** (oculto para usos de voz pura): soporte de bajos geométrico. La BR real depende mucho de materiales; este proxy mide sólo el aporte geométrico.
- **PRÁCTICO** (siempre): Forma (L/W y H/W con etiquetas “ok”, “túnel”, “pozo”, “pancake”), aprovechamiento de planta, constructabilidad.
- **ROBUSTEZ** (siempre): margen del α requerido respecto al rango razonable $[0,08; 0,30]$.

Cada sub-score aparece como un chip de color al final de cada sección: **verde** (≥ 80), **amarillo** (50–80), **rojo** (< 50).

19.4 Pesos condicionales por uso

Voz: Modal 40% + Voz 30% + Música 0% + Práctico 25% + Robustez 5%

Música: Modal 45% + Voz 0% + Música 20% + Práctico 25% + Robustez 10%

Mixto: Modal 40% + Voz 15% + Música 10% + Práctico 25% + Robustez 10%

Dentro de **MODAL**: 40% Bolt + 25% RT60-feas + 20% Modal-Q + 15% Schroeder. Bolt-spacing pesa más porque es la única métrica modal que discrimina la geometría real (las otras dependen mayormente de $V/\text{RT60}$ que son constantes para los 3 candidatos).

19.5 Aplicar un candidato

El botón «**Aplicar ▼**» de cada card abre un menú:

1. **Como parámetros (editable)**: setea los sliders de la pestaña Geometría y te lleva ahí.
2. **Como CAD (geometría fija)**: construye la malla y la inyecta como geometría externa.

Debajo del botón aparece una leyenda **Render**: 0,12 s · 6,0×8,2×4,3 m con el tiempo del último apply. La card de **Control Negativo** tiene «**Aplicar**» deshabilitado (con tooltip explicativo).

19.6 Evaluar tu diseño actual

Debajo de «**Predecir**» hay un segundo botón: «**Evaluar mi diseño actual**». Toma la geometría que tenés diseñada en la pestaña Geometría y la corre por el **mismo pipeline de scoring**. Útil para:

- Validar un diseño propio contra los 13 criterios objetivos.
- Comparar versiones iterando sobre los sliders.
- Saber si un CAD importado vale la pena antes de simular.

El resultado se muestra como **una card individual**. Si después querés ver alternativas, apretás «**Predecir**» y aparecen las 3 sugerencias para comparar visualmente.

Apéndice — Optimizaciones internas v2.3

Para usuarios técnicos que quieran inspeccionar el código:

Evaluación vectorizada del campo

Antes (loop Python en FieldEvaluator.evaluate_many):

```
def evaluate_many(self, field_nodal, points):
    out = np.full(len(points), np.nan, dtype=complex)
    for i, x in enumerate(points):
        e, N = _locate_one(self.v0, self.A_inv, self.tets, x)
        if e is not None:
            out[i] = complex(np.dot(field_nodal[self.tets[e]], N))
    return out
```

Para $\text{len}(\text{points}) = 62\,500$ y $N_e = 14\,400$, esto generaba 62 500 llamadas a `_locate_one`, cada una computando barycentric contra TODOS los tetraedros = $O(N_p \cdot N_e)$ bajo el intérprete de Python. En la práctica: **15–25 segundos** por refresh del campo.

Ahora (KDTree + numpy vectorizado):

```
# En __init__: precomputar centroides + cKDTree.
self._centroids = self.nodes[self.tets].mean(axis=1)
self._tree = cKDTree(self._centroids)

# En evaluate_many: para cada punto, K=12 candidatos via KDTree,
```

```
# despues barycentric vectorizada en arrays (Np, K, 3, 3).
_, cand = self._tree.query(points, k=12)
stu = np.einsum("pcij,pcj->pci", A_inv[cand], rel_pc)
# ... un solo argmax por punto, sin loops.
```

Resultado medido: **50–170× más rápido**, diferencia numérica $< 1 \times 10^{-15}$. El algoritmo es idéntico, sólo cambia cómo se busca el tet contenedor. Si más del 1% de los puntos no se localiza con $K = 12$ (puede pasar en bordes con tetraedros muy alargados), se reintenta con $K = 48$ solo para esos puntos.

Importación CAD con QProgressDialog

El flujo de import en `main.MainWindow._open_cad_import()` ahora:

1. Abre un `QProgressDialog` modal con `setMinimumDuration(200)` — sólo aparece si la importación realmente tarda.
2. Pasa un callback `progress(msg)` a `geom_import.load_geometry` para que mensajes de `gmsh/trimesh` aparezcan en vivo.
3. Mide cada fase con `time.time()` y al final reporta el desglose en la barra de estado: `load 230 ms · scale 5 ms · diagnose 410 ms · render 12 ms`.
4. Para mallas limpias (`diag.ok == True`) **salta el QMessageBox de confirmación**: antes había un «¿Usar esta geometría? Sí/No» que el usuario siempre aceptaba.

Prototipo 1 — Modelador de Recintos 3D con Simulación Acústica FEM

Manual de Usuario · v2.22 · 13 de Agosto de 2026

Cambios v2.0: import CAD (STL/OBJ/PLY/STEP/IGES/glTF), motor boundary-fitted con gmsh, router automático con badge UI, diálogo de reparación guiada, persistencia `.room` v3.

Cambios v2.1: router best-effort con fallback automático gmsh → voxel.

Cambios v2.2: diálogo de escala y orientación al importar (resuelve Y-up → Z-up de OBJ/glTF), reparador de mallas vectorizado en un solo pase (~ 50–100× más rápido) e indicador de ejes clicable con

atajo `Ctrl+Shift+Alt+X/Y/Z`.

Cambios v2.3: (1) materiales **por cara** con diálogo dedicado (estilo EASE, sección 6.3), persistencia entre aperturas y serialización en `.room` v4; (2) campo 3D 50–170× más rápido vía `cKDTree+numpy.einsum` (sección 18); (3) importación CAD con `QProgressDialog` y desglose de tiempos por fase; (4) suite de benchmarks `benchmark_v2.py` que produce `BENCHMARK_RESULTS.md`.

Cambios v2.4: diálogo de RT **multi-curva** con tres métodos de predicción (Sabine, Eyring, Fitzroy) y tres métricas (T60, T30, T20). Curvas agregables y quitables para comparar materializaciones distintas; cada curva guarda un snapshot numérico. Código de colores determinístico (azul/rojo/verde por método; sólida/discontinua/punteada por métrica). Sección 10.

Cambios v2.5: (1) diálogo de RT simplificado: se removieron Fitzroy y T20 / T30 de la UI (el código Fitzroy queda como `compute_fitzroy_rt60_per_face`); quedan Sabine y Eyring en T60; (2) botón «**Materiales**» sin emoji; (3) audio con boost de +6 dB equivalente vía soft-clipping tanh, fade-in 10 ms / fade-out 50 ms y cola de 100 ms de silencio para eliminar el *pop* al finalizar; (4) herramienta `check_materials_coverage.py` que cruza un listado externo de materiales contra la librería interna con matching difuso (stemming español + sinónimos en/es) y produce `MATERIALS_COVERAGE.md`; (5) bug fix: `UnboundLocalError` por sombreado de `fm` en `acoustic_panel._build_ui` (renombrado el `QFormLayout` local a `fmode`) y limpieza de la propiedad CSS `font-variant-numeric` no soportada por Qt5.

Cambios v2.6 (24–25 de mayo): (1) **Pestaña Predicción** con 3 candidatos (Bolt/Bonello/Louden) scoreados por **13 sub-scores** agrupados en 5 categorías (Modal, Voz, Música, Práctico, Robustez), pesos condicionales por uso (voz / música / mixto), control negativo automático cuando los 3 quedan parecidos (sección 19); (2) botón «**Evaluar mi diseño actual**» que aplica el mismo scoring a la geometría diseñada en la pestaña Geometría; (3) **auto-tuner de densidad FEM**: cuando el motor está en *Automático*, calcula la densidad necesaria para cubrir hasta $f_{\text{Schroeder}}$ dentro de un budget de tiempo (5 s shoebox, 10 s CAD/curvas) con diálogo de cobertura parcial si no entra; estimaciones recalibradas (voxel ~ 7 000 tets/s, safety factor 1,30); (4) skip de gmsh para techos curvos paramétricos (T-junctions); (5) **leyendas de tiempo** Último: X,XX s persistentes bajo botones de cómputo; (6) nuevos colormaps de la nube 3D: *rainbow* 7 paradas (azul-celeste-turquesa-verde-amarillo-naranja-rojo) para $|p|$ con fuente, y *signed vivid* azul-gris-rojo (gris en vez de blanco) para forma modal sin fuente; (7) bug-fixes: slice plane interactivo invisible para shoebox (z-fighting con paredes), auto-tuner desactivaba fallback gmsh, paneles con texto recortado en bordes.

Cambios v2.7 (25–26 de mayo): saneamiento profundo. (1) Vectorización del voxel mesher `points_inside_surface`: ~ 44× más rápido en mediana, Calcular modos (FEM) pasó de 1,3 s a 180 ms para shoebox típica; (2) eliminado el diálogo *cobertura parcial/completa/cancelar*: `auto_density` se llama siempre con `budget=∞`, cobertura completa hasta $f_{\text{Schroeder}}$ garantizada; (3) `QProgressDialog` pulsante durante el FEM con label dinámico por fase; (4) shift+drag de fuentes/receptor mucho más confiable (sincronización viewer↔panel arreglada, drag fantasma eliminado); (5) nuevo modo `Ctrl+Shift`+drag para mover en altura.

Cambios v2.8 (27 de mayo): (1) **Solver BEM removido por completo** del proyecto (`acoustic_bem.py`, `bem_modal.py`, `benchmark.py`, paper FEM/BEM, y todas las referencias cruzadas). El pipeline ahora es 100 % FEM; la función `schroeder_frequency` se mantiene como frontera campo modal / campo difuso. (2) **Export CSV/TXT en los tres diálogos de gráfico** (FRF, heatmap, RT60). CSV usa ; con coma decimal (Excel-es); TXT usa tabulador con punto decimal. (3) **Clamp constructivo** en el predictor: la altura de los candidatos se confina a [2,5; 4,0] m por defecto (13 hileras de ladrillo / jornada × 10 cm = 1,3 m/día). Activar «**Override altura**» si querés un techo de 10 m. Cuando hay clamp, W y L se reescalan preservando su proporción para mantener `V_target`. (4) **Fix UX import CAD**: el “Cargando” fantasma que aparecía ~ 3 s después del diálogo de escala ya no aparece (causa: `QProgressDialog.hide()` no cancelaba el `forceTimer`; ahora se usa `close()`). (5) El botón «**No escalar**» del diálogo de escala ahora sí saltea el escalado y **sigue** la importación (antes cancelaba

todo). Tres botones explícitos: «Aplicar escala», «No escalar», «Cancelar import».

Cambios v2.9 (28–29 de mayo): documentación interna profunda + robustez del solver FEM. (1) `acoustic_mesh_explicado.md` nuevo y `acoustic_fem_explicado.md` profundizado: explicadores línea-a-línea con cajas de `nodes[tets]` (fancy indexing) y *KDTree desde cero*, glosario de patrones NumPy/SciPy (broadcasting, einsum, `coo_matrix` con suma automática) y referencias bibliográficas. (2) `cuestionario_acoustic.html`: autoevaluación interactiva, **86 preguntas en 13 categorías** (física, mesh, ensamblaje, autovalores, barycentric, KDTree, FRF, NumPy, capciosas, código, numéricas) con desglose por categoría coloreado para identificar qué reforzar. (3) **Filtro de slivers** en `build_volume_mesh`: tetraedros con $V_e < 10^{-6} \cdot \bar{V}$ se descartan tras el filtro por centroide. Evita las entradas mal escaladas en K y M que son la causa principal de no-convergencia en Lanczos para mallas no axis-aligned. (4) `mesh_info` reporta tres claves nuevas: `h_min`, `h_ratio` (heterogeneidad: > 50 indica slivers o tets alargados) y `n_slivers` (umbral más laxo que el filtro, para detectar también casos borderline). (5) `solve_modes` robusto: try/except `ArpackNoConvergence` con tres planes (usar parciales si alcanzan / reintentar con $\sigma \times 10$ / `RuntimeError` con mensaje accionable que sugiere chequear `n_slivers`, reducir `n_modes` o cambiar `sigma`). `maxiter` subido de $10 \cdot N$ a $\max(300, 20 \cdot n_{\text{request}})$. (6) **Simetrización forzada** $K = (K + K^T)/2$, ídem M , al final de `build_KM`: cancela asimetrías residuales del orden de 10^{-15} que el scatter de `coo_matrix` puede dejar y que ensucian el shift-invert. Costo $O(\text{nnz})$, despreciable. Cero cambio en API pública; demo de caja $5 \times 4 \times 3$ m produce las mismas frecuencias y el mismo pico de FRF.

Cambios v2.10 (29 de mayo): evaluación de FEM P2 (elementos cuadráticos) — explorada, validada, y **descartada por costo/beneficio**. (1) Nuevas primitivas `shape=circle` / `ellipse` en `make_room`: planta circular o elíptica muestreada en 96 puntos por defecto (desviación $< 0,05\%$ vs curva real). Helper `sample_room_curve` expone la curva paramétrica para futuros consumers (visualizador, mallador isoparamétrico). (2) Implementación completa de **P2 subparamétrico** (10 nodos por tet, `shape functions` Lagrange en baricéntricas) con upgrade de malla $P1 \rightarrow P2$, cuadratura Keast 5-pt para K, y forma cerrada multinomial para M (la cuadratura introducía M_{ij} negativos por el peso central negativo del Keast 5-pt sobre integrandos de grado 4). (3) Implementación de **P2 isoparamétrico** con Jacobiano por punto de cuadratura, snap radial de midpoints a la curva real, y sanity check $\|K_{\text{iso}} - K_{\text{sub}}\|_F < 10^{-13}$ cuando los midpoints están en promedios aritméticos. (4) **Validación numérica**: caja $5 \times 4 \times 3$, P1 da error 0,4–3,2% vs analítico; P2 sub reduce a $< 0,05\%$ ($\sim 100\times$ mejora). En 5 salas $\times$ 30 modos, P2 cuesta $5\times$ a $36\times$ más tiempo que P1. (5) **Conclusión: P1 se mantiene como solver de producción**. El error de P1 con `n_per_meter=2` ya está por debajo del ruido del modelado físico (RT60 estimado, posiciones de fuentes, $\alpha(f)$ de materiales) y de la frecuencia de Schroeder; invertir $5\text{--}36\times$ más tiempo de cómputo para reducir un error que ya no afecta ninguna decisión acústica práctica no se justifica. Los archivos `acoustic_fem_p2.py`, `bench_p1_p2.py`, `bench_p2_iso.py` y `planP2.md` se removieron tras esta evaluación. El trabajo queda registrado en este changelog como evidencia de que la elección de P1 es deliberada y medida.

Cambios v2.11 (30 de mayo): auditoría física del solver modal. Dos resultados. (A) **Benchmark modal damping vs matriz C de impedancia** (`bench_modal_vs_impedance.py`). Shoebox $5 \times 4 \times 3$, $\alpha = 0,30$, `n_per_meter=2`, 12 modos. Modal damping pipeline 26 ms vs C-matrix directo 242 ms (**ratio** 9,5 $\times$; crece a $10^2\text{--}10^3\times$ con $Z(\omega)$ compleja). Ambos métodos localizan los mismos picos modales; en banda 30–100 Hz la diferencia es **RMS 1,6 dB** (dentro del ruido del problema). Conclusión: modal damping con $\xi_n = 1,1/(f_n \cdot \text{RT60})$ se queda como modelo de absorción de producción — la matriz C sólo ganaría con $Z(\omega)$ medida en tubo de Kundt, no con α de catálogo. Detalle en `acoustic_fem_explicado.md` §16. (B) **Fix de calibración: factor c^2 en la fórmula modal** (`frequency_response` y `modal_pressure_field` de `acoustic_fem.py`, y `frequency_response` de `fem_modal.py`). La derivación canónica de la Green function modal de Helmholtz da $p = i\omega\rho_0 c^2 \sum_n \varphi_n(x_r)\varphi_n(x_s)/(\omega_n^2 - \omega^2)$ (porque $\omega_n^2 = c^2\lambda_n$ y $k^2 = \omega^2/c^2$); el código histórico omitía el c^2 resultando en **toda la FRF y todos los campos de presión modales ~ 101 dB por debajo del SPL físico real**. No se había notado porque (i) la FRF se usaba para forma relativa, (ii) la auralización normaliza a peak antes del DAC. **Fix**: prefactor multiplicado por `c**2`; validado contra cálculo analítico (74,8 dB esperado vs 74,2 dB medido) y contra C-matrix (RMS 1,6 dB post-fix). Smoke test agregado a `acoustic_fem.__main__` (`assert SPL pico  $\in$  [50, 100] dB`) como guardia de regresión. **Compatibilidad rota**: exports CSV/TXT pre-v2.11 quedan +101 dB desfasados de los nuevos. Auralización inafectada.

Cambios v2.12 (30 de mayo): UX del solver modal + un bug físico de validez escondido. Seis ejes. (A) **Picker de modos con filtro $f_{\text{min}}/f_{\text{max}}$ y leyenda dinámica** debajo del combo Modo:. La leyenda

muestra total de modos calculados, rango real (f_0-f_n) y cuántos quedan filtrados. Si el filtro pasa por encima del rango calculado, lo avisa. Índice real preservado vía `userData` del combo — slice y heatmap siguen recibiendo el modo correcto aunque la posición en pantalla no coincida con el índice absoluto. (B) **Spinbox** Nº modos subido de (2, 80) a (2, 500) para permitir cobertura completa hasta `f_Schroeder` en salas chicas con f_S alta (un cuarto de 60 m³ con RT60 $\approx$ 1 s tiene $f_S \approx$ 270 Hz y necesita $\sim$ 170 modos por ley de Weyl). Label $\approx$ N modos hasta `f_Schroeder` (Weyl) debajo del spinbox. (C) **Sugerencia de n_per_meter** (compromiso D4): label npm sugerido: X.XX y botón [Aplicar] debajo del slider de densidad. El valor surge de $\text{npm} = \text{ppw} \cdot f_S / c$ y carga la malla exactamente válida hasta `f_Schroeder`. El slider sigue editable: preview rápido ignora la sugerencia, análisis riguroso la aplica. (D) **Fix de validez de malla**: `solve_modes` devolvía los N modos más bajos sin chequear si superaban $f_{\text{max_malla}} = c / (\text{ppw} \cdot h_{\text{max}})$. Modos arriba del techo de la malla son numéricamente sucios (dispersión, plegado de onda). Desde v2.12 hay un **post-clip automático** en el panel tras cada solve: los modos con $f > f_{\text{max_malla}}$ se descartan y se reporta al log ("pediste 256 modos, 210 son válidos, 46 excedían..."). (E) **Fixes estéticos**: botones largos del diálogo de importar/reparar CAD (✓ Cerrar este hueco, etc.) se recortaban por el clipping de texto centrado con `sizeHint` subestimado por el Unicode al inicio. Triple defensa: `minimumWidth` del panel (440 px), `minimumWidth` por botón (380 px), y `text-align: left; padding-left: 16px` via `styleSheet` local. Botones Exportar PNG/SVG/PDF/CSV/TXT en FRF, RT60 y Slice heatmap bumpeados a `minimumWidth` 140. (F) **Lo que no cambió**: solver FEM intacto (cambios sólo en panel UI y post-procesamiento), API pública intacta, decisión D4 vigente, auralización igual.

Cambios v2.13 (junio): batch grande post-v2.12 — criterios de diseño bibliográficos, fuentes reales, geometría no-prismática, optimizador de ubicación de fuentes y diagnóstico de corregibilidad por EQ. Once ejes. (A) **Fuentes con respuesta $Q(f)$ + fase**: cargan mediciones FRD (`frd.py`); la respuesta es una ganancia compleja $g(f)$ relativa al Q baseline (sin curva $\rightarrow$ FRF idéntica, calibración c^2 intacta); anclaje absoluto/relativo, atajo delay+polaridad+offset de fase; `SourceResponse` en `sources.py`, UI en `SourceEditDialog`, persistencia `.room` v5. (B) **Ratios**: corrección de nombres en `RATIO_LIBRARY` (Louden/Bolt/Sepmeyer) + Cox (1:1.56:1.86); `predict()` recorta al top-3. (C) **Altura por uso** en `USE_PRESETS` (3–12 m según uso), sin el cap duro de 4 m, con override del usuario. (D) **Geometría loftada**: recinto no-prismático (`make_lofted_room`), wizard de cortes laterales (`section_dialog.py`), `.room` v6, watertight. (E) **Baffle orientado** (azimut/pitch/montaje, puramente visual: la fuente sigue omni), wireframe en el viewer 3D, gesto Alt+Ctrl+arrastrar. (F) **SBIR** (`sbir.py`): fuentes imagen de 1er orden, dB relativo al directo, notches en $c/(4d)$; `SBIRDialog`. (G) **Métricas de respuesta forzada** (`modal_metrics.py`): `FoM_flat`/`FoM_espacial` (planitud media y consistencia asiento-a-asiento sobre una grilla, banda válida) y cruce modal numérico `f_cross` (estilo MDCF, ve la forma); cableadas a la UI. (H) **Optimizador de ubicación de fuentes** (`location_opt.py`): modo de Predicción

Geometría/Ubicación/Combinado, objetivo FoM+SBIR+suavidad modal con pesos por uso. (I) **Criterios al score**: minado bibliográfico (`criterios_room_geom_fuente.md`) $\rightarrow$ FSI $\psi(25)$ de Rindel (A6) y densidad Bonello (A3) al grupo MODAL; damping per-modo por cara (A36), Bass Ratio (D5), umbral de Fazenda (C9, reemplaza el $Q > 30$ fijo). (J) **Corregibilidad por EQ** (`eq_correctability`): clasifica por frecuencia qué arregla un EQ global y qué exige acústica, vía consistencia espacial (cota irreducible `fom_espacial`, invariante a EQ global) + envolvente sin cancelación (fase mínima sin cepstrum); loop cerrado que mide `improvement_flat`; sub-banda confiable ($\text{ppw} \geq 15$); overlay rojo en la FRF. Validado vs teoría y vs Welti 2003. (K) **Fix del auto-tuner de malla** (`mesh_router`): presupuesto infinito y sub-entrega de validez de `gmsh` ($h_{\text{max}} \approx 1,5 h_{\text{target}}$); 81 $\rightarrow$ 308 modos válidos. (L) **Lo que no cambió**: solver FEM intacto, decisiones D1/D2/D4/D5b vigentes, calibración c^2 y API pública intactas.

Cambios v2.14 (27 de junio): features post-batch v2.13. (A) **Undo/redo GLOBAL** (Ctrl+Z / Ctrl+Y, 10 acciones): deshace *cualquier* acción (fuentes, receptor, materiales, CAD, geometría, aplicar predicción) por **snapshot del estado completo** reusando la serialización `.room`; un timer de polling ($\sim$ 400 ms) hace dirty-check $\rightarrow$ global por construcción, sin instrumentar cada mutación. Drag = 1 acción (settle); snapshot = inputs, no resultados FEM; límite 10. En `main.py` (`_capture_state` / `_restore_state`). (B) **Gate de materiales en Predicción**: al Predecir sin elegir absorción, aviso con 3 caminos — que elija el programa (RT por uso), *preset* piso madera/paredes ladrillo/techo madera, o coeficiente α uniforme. En preset/uniforme los materiales **determinan el RT60 por candidato** (Sabine hacia adelante, `effective_rt60` en `prediction.py`). La asignación fina por cara sigue en Acústica. (C) **Fix**: labels de diagnóstico de la FRF a blanco (eran ilegibles sobre el fondo oscuro). (D)

Editor de forma: edición numérica exacta. En la planta (`shape_dialog.py`) cada arista muestra su **longitud** en un chip clickeable (click → valor exacto; el primer vértice queda fijo y el otro desliza en la dirección de la arista) y el origen (0,0) se corre del centro a la **esquina inferior-izquierda** (cuadrante positivo). En los cortes laterales (`section_dialog.py`) cada punto del perfil muestra su **altura** clickeable, y la relación con la pared opuesta pasa a un selector *Libre / Espejo / Igual* (espejo en t vs copia directa). (E) **Sistema de presets de materiales** (compartido Predicción+ Acústica, en `material_library.py`): presets con nombre (Reflectante / Estudio tratado / Home theatre / Aula / Neutra) que mapean piso/paredes/techo a materiales reales del catálogo (α por banda), resueltos por nombre sin acentos. En Predicción el gate de absorción ofrece preset + armar-el-tuyo (3 combos) y el RT60 por candidato sale de esos materiales (`effective_rt60` modo materiales); el botón “Aplicar a Acústica” asigna por zona (deshacible). En Acústica, botón “Preset nombrado”. Fix: word-wrap en el status del main (un nombre largo estiraba y cortaba el panel). (F) **Documentado** (sin código nuevo) un fix de fondo del router de mallado: la validez la define el peor tet ($h_{\max}$); gmsh entrega $h_{\max} \approx 1,5 h_{\text{target}}$, así que el auto-tuner le pide a gmsh una malla $1,5\times$ más fina (`_GMSH_HMAX_OVER_HTARGET`) para que la validez real alcance f_S en vez de $f_S/1,5$; un guard `np.isfinite` evita que el tuner muera con `budget = ∞`. Verificado en GUI (voxel valida $\approx f_S$; gmsh $\geq f_S$). Ver § “Cálculo de modos FEM”.

Cambios v2.15 (30 de junio): correcciones de la pestaña Predicción al evaluar el diseño propio, geometría irregular en la evaluación, y dos fixes de UI. (A) **“Evaluar mi diseño actual” respeta el criterio:** antes scoreaba siempre la geometría ignorando el combo *Optimizar* y las fuentes; ahora respeta el eje — Geometría (la forma), Ubicación (tu layout REAL de fuentes vía `evaluate_layout`, no optimiza) o Combinado. Lee las fuentes del recinto (callback `get_sources`); sin fuentes avisa (Ubicación) o degrada a geometría (Combinado). `prediction.evaluate_design`. (B) **Geometría irregular:** evaluar una planta dibujada (`base_polygon`) o con cortes (`wall_profiles`) reconstruía una caja con `make_room` e ignoraba la forma; ahora el FEM corre sobre la malla REAL renderizada (callback `get_surface`). Como el score de geometría es de cajas, un diálogo ofrece *aproximar por la caja envolvente (AABB)* o *no ponderar la forma* (que impide predecir por geometría). Corrige además el aviso espurio de “fuentes fuera del recinto” al evaluar una forma dibujada. (C) **Slider de Alto:** con forma personalizada se bloqueaban Ancho y Largo pero no el Alto; ahora el Alto se deshabilita cuando hay cortes laterales (la altura la definen los perfiles) y queda editable con solo-planta. `controls.py`. (D) **Receptor reubicado:** el editor de forma corre el origen a la esquina inferior-izquierda, así que el receptor por defecto en (0,0) caía fuera de una planta dibujada y disparaba el aviso al calcular modos; ahora, al cambiar la geometría o cargar un `.room`, si el receptor está fuera se reubica a un punto interior. `acoustic_panel._relocate_receiver_if_outside`. (E) **Tooltips legibles:** regla `QToolTip` (fondo blanco, texto negro) — antes heredaban texto claro sobre el fondo amarillo de Qt. `style.py`.

Cambios v2.16 (5 de julio): origen configurable, análisis multi-punto con mute por fuente, mediciones `.trf`, f_S desde materiales, y fixes de Predicción y estabilidad. (A) **Origen (0,0,0) configurable** (combo en Dimensiones): Auto (convención histórica de cada camino) / Centro de planta / Esquina inf.-izq., para paramétrico, planta dibujada y CAD. Fuentes, receptor y puntos de escucha se trasladan CON el recinto (solo cambia el sistema de coordenadas); se guarda en el `.room`.

`geometry.origin_offset/anchor_vertices`; `bench_origin_mode.py`. Fix asociado: el loader re-centraba el CAD embebido incondicionalmente y pisaba el frame guardado; ahora re-ancha según el `origin_mode` del archivo. (B) f_S **desde materiales** + panel propio entre Materiales y FEM: punto fijo $f_S = 2000\sqrt{RT(f_S)/V}$ interpolando el RT60 de Sabine por bandas de los materiales asignados ($\alpha=0,05$ solo como fallback; el log dice qué usó). (C) **Hover en Materiales:** la fila bajo el cursor resalta sus caras en el 3D (aditivo, visible aun ocluido). (D) **Carga de `.trf`** (transfer function binaria, firma JACKREF!; magnitud/fase/coherencia, eje MTW): detección por contenido, anclaje auto-*Relativo* (la TF es relativa, no SPL), aviso si la coherencia mediana $< 0,7$. `frd.load_trf`; `bench_trf.py`. (E) **Mute por fuente:** checkbox en la lista; silenciada no radia (FRF/SBIR/FoM/campo 3D/Comparar) pero conserva todo; persistido (`active`). (F) **Puntos de escucha nombrados** (Sweet Spot, Mic N): lista en el grupo Receptor, esferas cyan en 3D, persistidos, doble click mueve el receptor. (G) **Comparar...**: tabla FoM por posición (planitud local + desvío vs promedio; fila CONJUNTO con $VSA = \sigma_f$ del promedio espacial y $MSV =$ media del σ entre posiciones, sobre las posiciones REALES del usuario) + curvas FRF y SBIR superpuestas por punto, con las fuentes activas; export CSV/TXT/PNG. (H) **Marcadores en el heatmap 2D:** fuentes activas o y receptores $\times$ con nombre debajo (blanco con borde negro); semi-transparente si está a $> 0,5$ m del plano de corte. (I) **Predicción de ubicación irregular:** el FEM corre sobre la malla real (leyenda “malla real”); las semillas del AABB que caen fuera de la sala se

REPARAN por bisección hacia un ancla interior (`LocationContext.inside_fn/repair_layout`). (J) **Estabilidad**: fix del freeze al silenciar (rebuild de la lista desde su propio evento colgaba el mouse-grab de Qt); puntos de escucha con esferas `GLMeshItem` en vez de scatter GL persistente; watchdog opcional `PROTO1_WATCHDOG=1` (stacks en consola si la GUI se cuelga > 20 s).

Cambios v2.17 (14 de julio): parches de absorción sub-cara y carga de materiales propios desde JSON. (A) **Parches de absorción sub-cara**: botón «**Parches de absorción...**» + editor 2D para dibujar regiones DENTRO de una cara con su propio material. Da resolución sub-cara al amortiguamiento modal selectivo (A36): el α del parche entra por el RT60 de Sabine (restando área al anfitrión) y por ξ_n pesado por ϕ_n^2 sobre la región. Las ϕ_n se calculan con paredes rígidas, así que un parche NO cambia la forma modal ni el heatmap; el efecto es $\xi_n \rightarrow RT \rightarrow FRF$. *Cuadratura*: sin parches se usa A36 crudo (los `.room` sin parches no cambian ni un dígito); con ≥ 1 parche se conmuta a cuadratura FINA (más precisa que la malla gruesa: los números pueden moverse, no es un error). Reduce exacto a A36 con material uniforme. `absorption_patch.py`; `.room v8` (`absorption_patches`); `bench_absorption_patch.py`. (B) **Editor 2D**: modo Rectángulo (arrastrar) y modo Polígono (click por vértice, convexo o no; cerrar cerca del primer punto / `[Enter]` / doble click; botón derecho o `[Esc]` deshace); rueda = zoom centrado en el cursor; snapping a grilla; **sin solapes** (candidato en rojo, no se agrega; se permite adyacencia por arista). Geometría en `absorption_patch`: `poly_area` (shoelace), `points_in_poly` (ray casting), `triangulate_uv` (ear clipping, para no convexos), `polys_overlap`. `patch_dialog.py`. (C) **Overlay 3D**: los parches se pintan sobre la cara con el color de su material, con el quad separado 4 cm hacia el interior. `IsoViewer.set_patches` crea un `GLMeshItem` por parche con color uniforme. *Gotcha*: un `GLMeshItem` con `shader=None` + `faceColors` NO renderiza en esta escena, y no es cuestión del modo de profundidad (`translucent`, `additive` y `opaque` fueron los tres invisibles); ya estaba documentado en `acoustic_viewer.SourceMarkers`. El único patrón probado es el de `set_highlight_faces`: color uniforme + `shader=None` + `glOptions='additive'`. (D) **Parches en el diálogo Materiales**: se listan debajo de las caras (Parche (rect/polígono) en *cara*) con área y categoría; su **combo de material es editable** (recolora el overlay en vivo y recalcula ξ /RT al aplicar); el **hover** sobre la fila resalta el parche en el 3D (`set_highlight_patch`, ítem propio que no pisa el overlay permanente). (E) **Cargar tu material** (JSON): cuadro con la sintaxis + selector de archivo; valida (nombre + absorción por banda), copia a `materials/` sin pisar el catálogo y recarga la biblioteca en el sitio (`MaterialLibrary.reload`) — disponible en todo el programa sin reiniciar.

Cambios v2.18 (19 de julio): **muebles** como obstáculos en el modelo modal — modelado completo (obstáculo rígido + absorción + reflexión) y manipulación directa en el visor (uso en §6.4). (A) **Obstáculo rígido (carve)**: un mueble (caja o cilindro) talla la malla del aire (los tets adentro se quitan antes de `build_KM`; la superficie del hueco queda rígida por condición natural). Los modos se corren solos, exacto. Preserva la malla original para la absorción; API estable del solver intacta (`furniture.carve_mesh`; `muebles=[]` → idéntico al histórico). (B) **Absorción (A36)**: con material asignado, las caras aire-mueble entran al mismo A36 que las paredes (ξ_n pesado por ϕ_n^2 sobre el mueble); sin material = rígido. Un sillón tapizado debe absorber. (C) **Reflexión (SBIR)**: la cara superior del mueble se agrega al SBIR como panel finito (rolloff de Rindel por difracción de borde). (D) **Interfaz**: grupo «**Muebles**» (Añadir/Editar/Quitar/Duplicar) + `FurnitureEditDialog` (tipo, centro, tamaño, yaw, pitch, material, etiqueta) + **wireframe** verde-azulado en el visor (`GLLinePlotItem`, nunca `GLMeshItem(shader=None)`). Persistido en el `.room` con su material (`furniture_materials`, aditivo). (E) **Manipulación directa** (mismos gestos que las fuentes): `[Shift]`+arrastrar (mover XY), `[Ctrl]`+`[Shift]` (Z), `[Alt]`+`[Ctrl]` (girar yaw horizontal / inclinar pitch vertical), doble-click (editar). El **pitch es físico**: afecta el carve, no es solo visual (pitch=0 reduce exacto). (F) **Colisiones (AABB)**: un mueble no se superpone con otro mueble ni con el baffle de un parlante, ni se sale del recinto (frena en el drag; avisa en Añadir/Editar). Además, fuentes y receptor se traban en las paredes del recinto al arrastrarlos (clamp al bounding box).

Cambios v2.19 (29 de julio): **presets de muebles armados** (con forma) y fixes de manipulación del test visual. (A) **Muebles compuestos**: nuevo tipo *compound* (unión de sub-piezas en frame local); el `contains` es la unión, así un mueble con forma se talla/mueve/rota/choca como una sola pieza. La forma es física (afecta el carve); las partes finas no resuelven en la malla (correcto). `contains/aabb/volumen/persistencia/wireframe` despachan por tipo; caja y cilindro reducen exacto. (B) **27 presets** en menú agrupado (General 7 / Aula 10 / Estudio 10): silla... biblioteca + pupitre, pizarrón, casilleros... + gobos, bass traps, difusores QRD, resonadores Helmholtz, nubes, console desk, racks, sofá de control. Material sugerido del catálogo y placement (nubes al techo). *Alcance*: la difusión

(QRD) y la sintonía Helmholtz NO se simulan (FEM LF modal) → geometría + material aproximado; los absorbentes de banda ancha sí. Detalle en §6.4. (C) **Picking por silueta**: el mueble se agarra clickeando en cualquier parte de su bounding box proyectado (antes solo cerca del centro → los grandes no se dejaban agarrar). (D) **Rotación solo yaw**: Alt+Ctrl gira solo azimut; antes el arrastre vertical lo inclinaba sin querer (el mueble “se caía” y se trababa). El pitch se edita por el diálogo. El piso ya no atrapa al mueble.

Cambios v2.20 (30 de julio): **importar muebles desde CAD** y **gizmo de rotación de 3 ejes**. (A)

Mueble desde CAD: nuevo tipo *mesh* (.obj, .stl, .ply, .off, .glb, .gltf); el carve usa un test punto-adentro sobre la malla, así la pieza importada afecta modos, absorción y SBIR igual que un preset.

Caso de uso: escanear un estudio real, separar las piezas en SketchUp y exportarlas una por una. Al importar se repara la malla si hace falta (unir vértices, tapar huecos, normales) y se avisa si queda abierta (el test punto-adentro solo es confiable con superficie cerrada). La malla se embebe en el .room. Detalle y alcance en §6.4.1. (B) **Gizmo de rotación**: manteniendo Alt+Ctrl aparecen tres anillos (yaw celeste, pitch ámbar, roll verde); el anillo bajo el cursor se resalta y al arrastrar el mueble gira solo sobre ese eje. El eje se elige *antes* de mover, con click explícito → no se puede inclinar sin querer al rotar. Los anillos usan los mismos ejes locales que el tallado. (C) **Tercer eje (roll)**: vuelca el mueble de costado (gira sobre su frente). Convención aviación (yaw → pitch → roll). Como el pitch, *afecta el carve*. roll=0 reduce exacto y los .room previos cargan con roll nulo (sin bump de versión). (D) **Fix: se trababan al duplicar**: la copia nacía en la posición exacta del original → solape del 100% y ningún movimiento posible en ninguno de los dos. Ahora la copia nace desplazada, y un mueble ya solapado puede arrastrarse para afuera (antes solo frenaba, nunca escapaba). Afectaba también a los presets.

Cambios v2.21 (31 de julio): **parches con espesor** (prisma) y **reglas de superposición** entre objetos de la escena. (A) **Parche como prisma**: se dibuja con el espesor real del tratamiento (10 cm por defecto) hacia el interior de la sala, con las aristas resaltadas para leerlo con cualquier color de relleno. Espesor 0 → plano, como antes. (B) **Espesor desde el material**: al elegir un material se autocompleta leyendo la construcción del nombre (suma capa absorbente + cavidad + cámara de aire + descuelgue; descarta la geometría en-plano de los paneles ranurados). Cubre 240 de 429 materiales; avisa si el espesor dibujado no coincide con el del α elegido, y muestra el $\lambda/4$. *El espesor NO entra al solver*: el $\alpha(f)$ del catálogo ya se midió con ese espesor (misma lana, α a 63 Hz cambia $\sim 15\times$ entre 20 y 100 mm) → sumarlo sería contarlos dos veces; y correr la frontera sería peor (a 34 Hz un panel de 10 cm es $\lambda/100$: la onda lo atraviesa). Detalle en §10.5. (C) **Fuente y receptor nunca dentro de un mueble**: el mueble se modela quitando el aire, así que un punto ahí adentro evaluaba NaN y contaminaba toda la FRF sin dar error. Se bloquea en los dos sentidos (mover el punto al mueble, y poner el mueble sobre el punto). El receptor estaba especialmente expuesto por ser un punto sin baffle. (D) **Regla de sólidos simétrica**: el baffle del parlante ahora también frena contra los muebles (antes solo frenaba el mueble contra el parlante, aunque el manual ya prometía lo contrario). Se mantiene el escape: lo que ya está superpuesto se puede arrastrar para afuera. (E) **Aviso de mueble tapando un parche**: no bloquea (el prisma es dibujo), pero un mueble delante del absorbente lo tapa y su α efectivo baja respecto del catálogo. (F) **Fix: muebles y parches se mueven con el origen**: al cambiar la convención de origen se trasladaban fuentes, receptor y puntos de escucha, pero muebles y parches se quedaban en el lugar viejo (el origen configurable es de v2.16; parches y muebles se agregaron después).

Cambios v2.22 (13 de agosto): **empaquetado del .exe puesto al día** (sección §20 nueva). (A)

Bundle 490 MB mas liviano: build.bat excluye lo que Anaconda arrastra y el proyecto no usa (Qt5WebEngineCore 107 MB, panel 101, botocore 92, llvmlite/numba 66, bokeh). Carpeta 1524 → **1032 MB**, ZIP 570 → **414 MB**, sin perder ninguna función. Lo que queda es irreducible: mkl_*.dll (~370 MB, BLAS de numpy/scipy con variantes de despacho por CPU), gmsh (~86 MB) y PyQt5 (~81 MB).

(B) **El ZIP dejo de mentir su version**: el nombre estaba hardcodeado en v2.12 y quedó congelado ocho versiones; ahora pack_distribution.py lo lee del changelog de este manual. (C) **§20 nueva + verificador al día**: flujo completo (compilar, verificar, smoke test, prueba visual, empaquetar), desglose del tamaño y *por que el ejecutable NO va al repositorio* (GitHub bloquea >100 MB y el historial queda inflado para siempre → la vía correcta es GitHub Releases). El rango de verify_distribution.py estaba en 100-800 MB y marcaba FAIL en un bundle correcto; ahora 700-1400 MB. Además -collect-all para trimesh y gmsh: no corregía un bug (PyInstaller ya los detectaba) pero los dos resuelven cosas en runtime que el análisis estático no ve. (D) **Fix crítico: el .exe salía sin las DLLs de Qt**. Anaconda no guarda las DLLs de Qt junto a PyQt5 sino en <env>\Library\bin\ y con otro nombre (Qt5Core_conda.dll); PyInstaller las busca recorriendo el PATH, así que al compilar desde una

consola sin el entorno conda activado *las omitia en silencio* — el build decia “BUILD OK” y el programa moria al abrirlo con `DLL load failed while importing QtCore`. `build.bat` ahora arma ese PATH por su cuenta. Los dos chequeos automaticos tampoco lo detectaban: `verify_distribution.py` solo miraba que existiera la CARPETA `PyQt5/` (existia, con los `.pyd`; faltaban las DLLs) y `test_distribution_smoke.py` daba OK con “el proceso sigue vivo 15 s” — pero un dialogo de excepcion tambien es un proceso vivo. Ahora el primero exige `Qt5Core/Qt5Gui/Qt5Widgets` y el segundo lee el titulo de las ventanas del proceso.